GENERATIVE AND AGENTIC AI · TOPICS 5 & 6

Deploying & Evaluating LLMs

Serving models in production, then measuring their quality

Laura Wynter, PhD
SMU School of Computing and Information Sciences
IS469 · AY2026–27

ROADMAP

What we'll cover

01 Part 1, Deploying LLMs for Inference

02 Decoding, KV cache & batching

03 Serving stacks & local deployment

04 Part 2, Evaluating LLMs

05 Classical metrics & perplexity

06 Benchmarks, LLM-as-judge & pitfalls

01

How Generation Works

Every response is a loop over next-token predictions. When it stops, and what it picks, are choices.

STOPPING

How does generation stop?

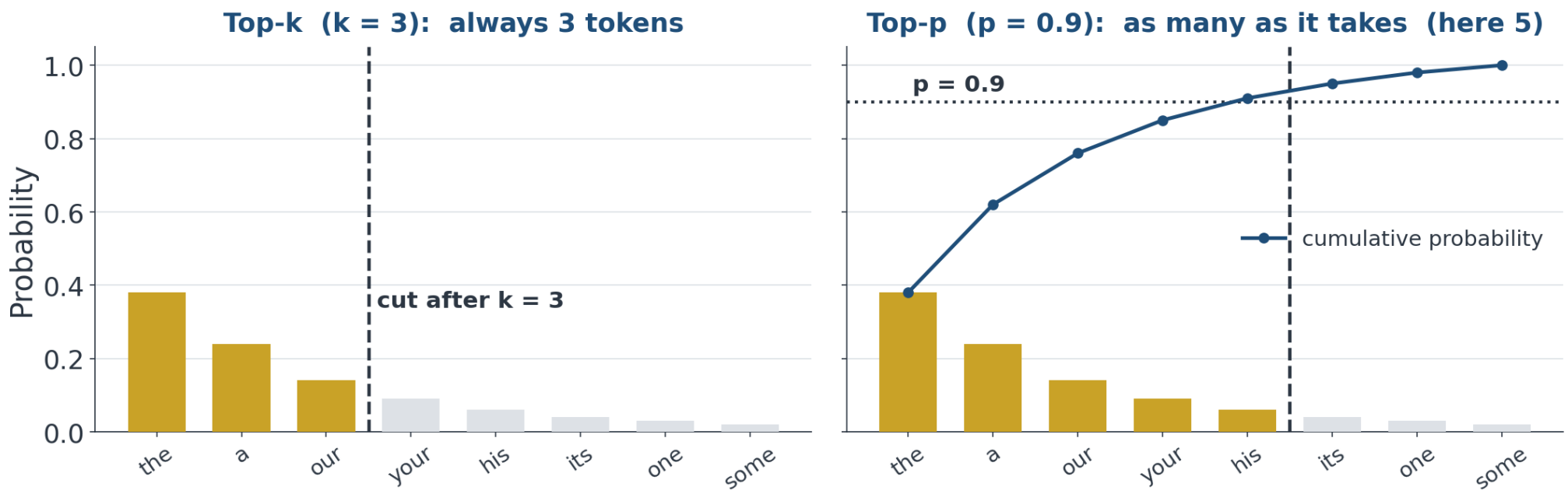
- EOS (end-of-sequence) token: a special token that means "stop generating"
- During training every text ends with EOS, teaching the model where the natural stopping point is
  - Prompt "What is the capital of China?" → model learned to emit "Beijing EOS"
- Max token length: a hard cap on the response, regardless of EOS
- LLMs front-load the summary / most important content, then add detail up to the limit

Selecting the next token from the softmax

- Greedy: always take the highest-probability token, consistent, but repetitive and less creative
- Top-k sampling: sample from the k most-likely tokens, larger k = more creative, smaller k = more consistent
- Top-p (nucleus): take the smallest set of top tokens whose cumulative probability reaches p (e.g. 0.9)
- Beam search: keep the k most promising sequences at each step (illustrated with beam width = 2)
- Sampling (top-k / top-p) is the default for open-ended text; beam search for constrained, single-best outputs

What top-k and top-p actually cut

Gold = tokens you can sample from. Top-k is a fixed count, top-p adapts to the shape



Same distribution, two rules. Top-k always keeps a fixed number of tokens; top-p keeps however many are needed to reach probability mass p, so it widens on flat distributions and narrows on confident ones.

DECODING STRATEGIES

Choosing the next token, four strategies

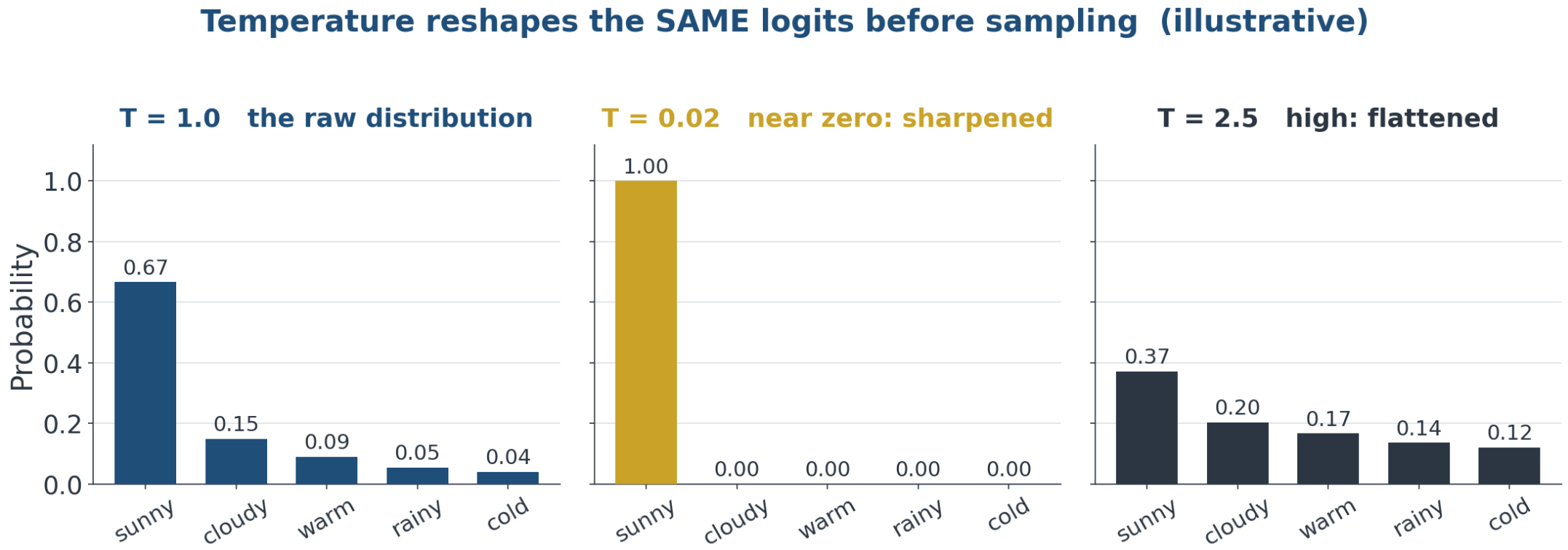
Strategy	What it does	Best for
Greedy	Always take the single highest-probability token	Deterministic, factual
Top-k	Sample from the k most-likely tokens	Tunable creativity
Top-p (nucleus)	Sample from the smallest set reaching cumulative p	Open-ended text
Beam search	Keep the k most promising whole sequences	Constrained, single-best

Greedy & beam are deterministic; top-k / top-p add controlled randomness, often with temperature.

Temperature, scaling the softmax

- Softmax: $\sigma(z)_i = \exp(z_i) / \Sigma_j \exp(z_j)$
- Temperature T rescales the logits: $\sigma(z, T)_i = \exp(z_i / T) / \Sigma_j \exp(z_j / T)$
- $T > 1$: flattens the distribution toward uniform, higher entropy, more creative / random output
- $T < 1$: sharpens it, the most confident predictions dominate, lower entropy, more predictable output
- Temperature is used jointly with top-p (nucleus) sampling to shape the final distribution

Temperature sharpens or flattens the curve



The same logits, three temperatures. T≈0 collapses onto the top token (greedy); T=1 is the model's own distribution; high T flattens it toward uniform, so unlikely tokens get sampled.

LAB

Hands-on, temperature & sampling

- Topic5-Temperature.ipynb, distilGPT2: play with temperature, top-k and top-p sampling
- Watch how the settings change generation, and confirm temperature ≈ 0 collapses to greedy
- distilGPT2 is a small, pedagogical stand-in for the frontier models we deploy in practice

02

Inside a Forward Pass

Prefill and decode are two very different phases, and the KV cache is why decode is cheap.

TWO PHASES

Inference runs in two phases

Tokenise prompt
text → token IDs on CPU, sent to GPU

→

Prefill
process the whole prompt in one parallel pass → first token

→

Decode
generate one token at a time, reusing the KV cache

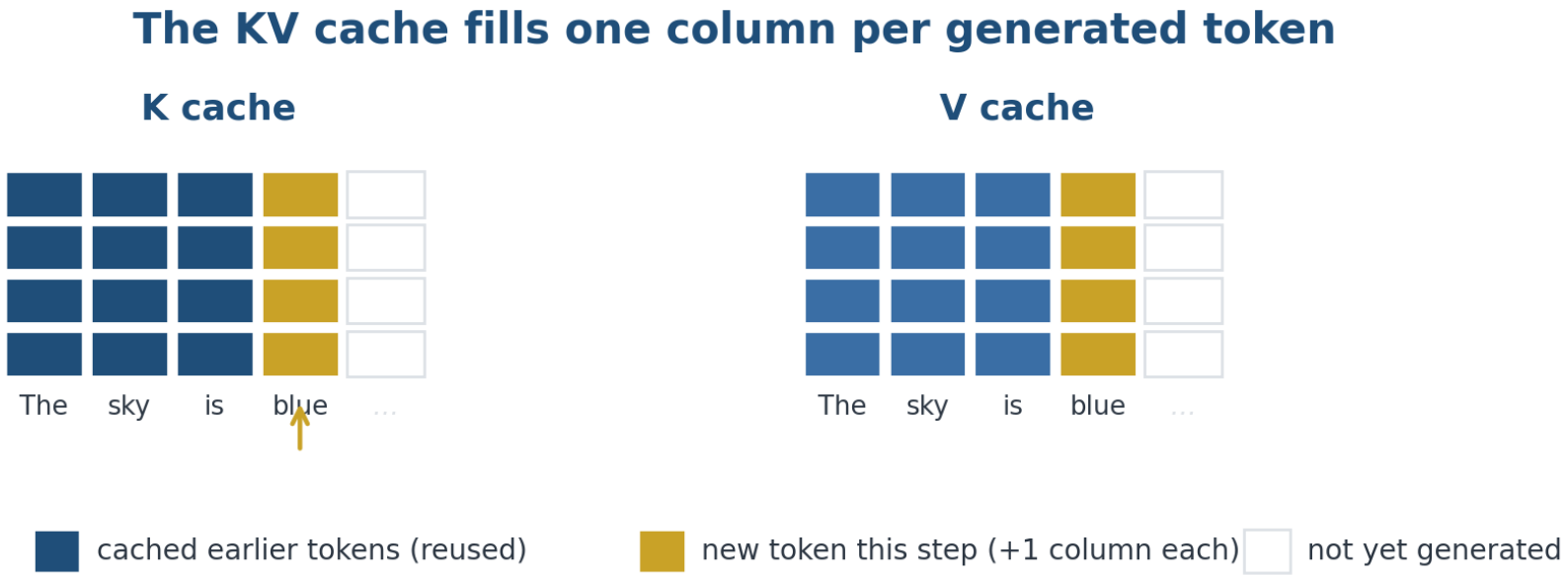
Prefill is parallel over all prompt tokens; decode is strictly sequential, one token per step.

TWO PHASES

Prefill and decode

- Prefill has three steps:
  - Load model weights onto the GPU
  - Tokenise the prompt on CPU, transfer the tensor to the GPU
  - Run the whole prompt through one forward pass to generate the first token
- Decode: append the just-generated token to the sequence and forward it to produce the next token, repeat
- Prefill processes all prompt tokens in parallel; decode is strictly one token at a time

Prefill fills the columns, decode adds one per step

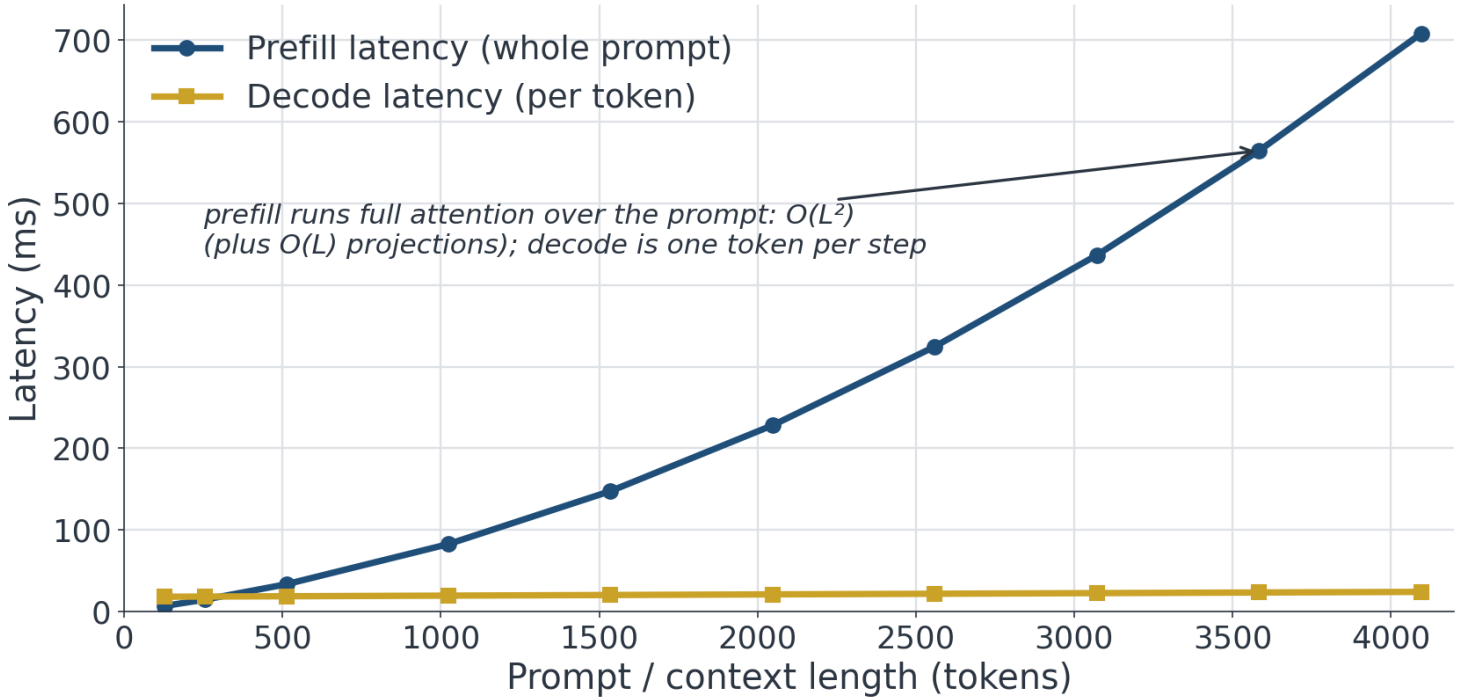


Per-token K/V size is fixed → the cache grows one column per token → memory is linear in sequence length.

Prefill computes K and V for every prompt token in one parallel pass (the blue columns). Each decode step then appends exactly one new column (yellow) and reuses the rest, which is the KV cache explained next. Illustrative.

Two phases, two latency profiles

Prefill grows quadratically with prompt length; decode stays flat (illustrative)



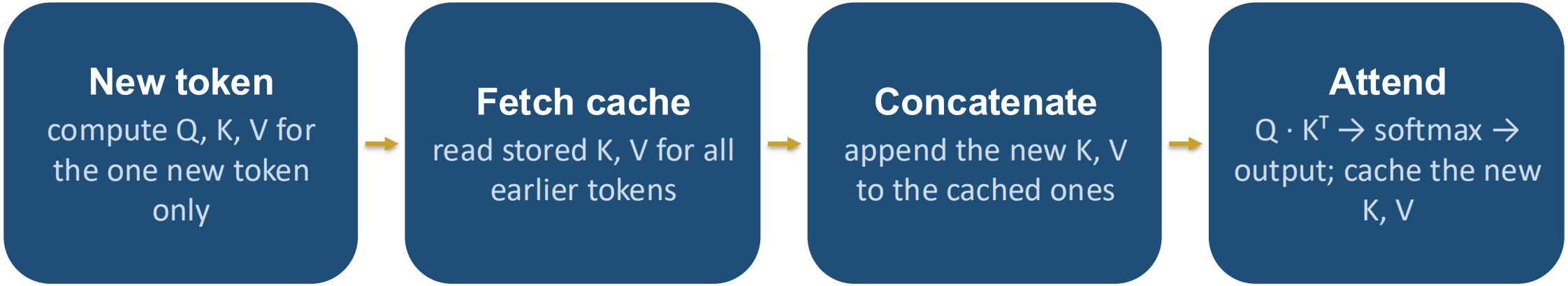
Prefill runs full attention over the whole prompt in one pass, so its cost grows quadratically with prompt length; per-token decode latency stays roughly flat. Illustrative.

WORKED EXAMPLE

Why we cache K and V

- Prompt "What color is the sky?", already generated "The sky is", now compute the vector for "is"
- Need Query for "is", plus Key & Value for every earlier token to compute attention
- But K and V for all earlier tokens were already computed on the previous step, recomputing is waste
- Idea: cache K and V from past tokens; each step only compute Q, K, V for the one new token
- Fetch cached K, V, concatenate the new token's K, V, then attention scores → output vector for "is"

One decode step with the KV cache

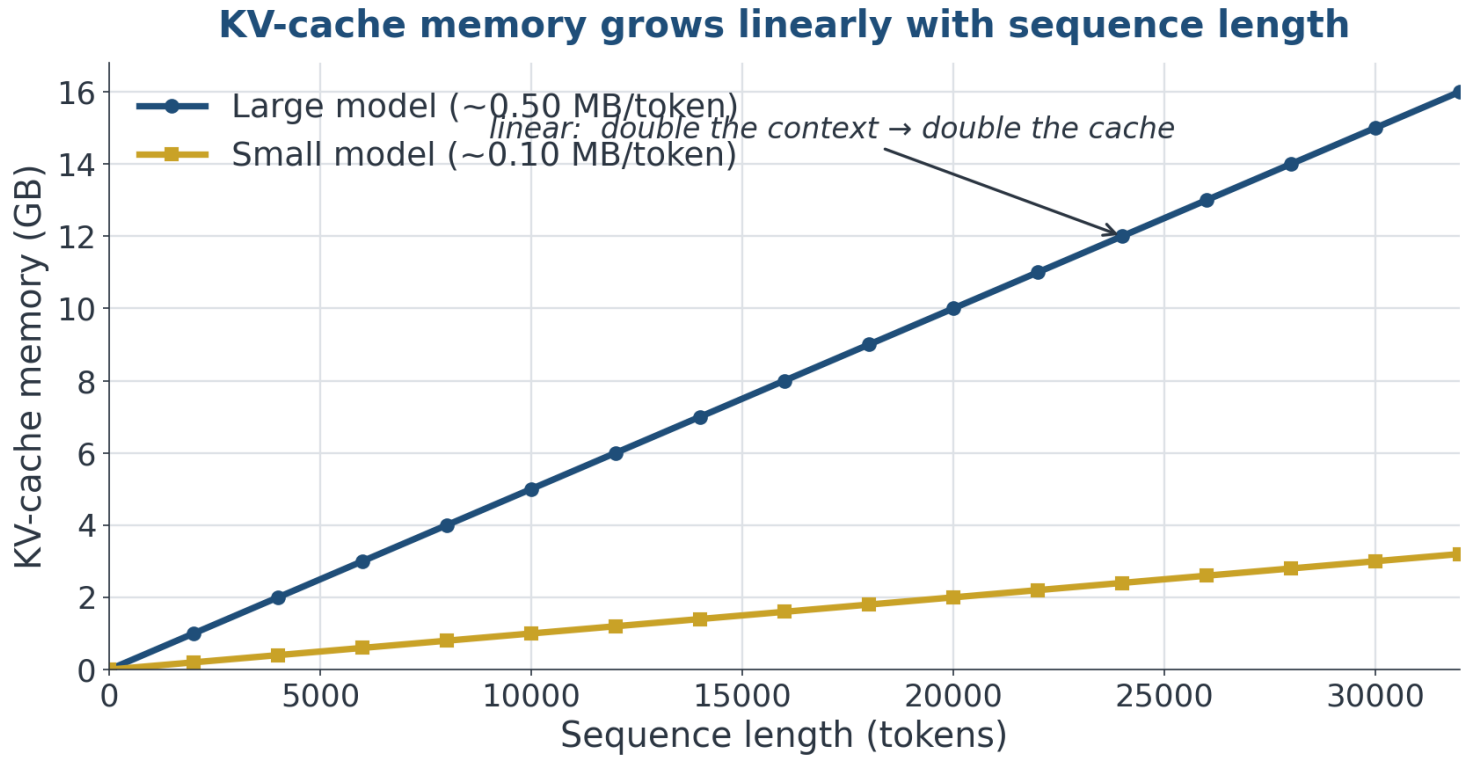


Recap of the picture above: only the new token is projected each step, past K/V are reused, never recomputed.

The KV cache gives linear scaling

- To generate the next token the model attends over all previous tokens, naïvely, repeated work every step
- KV caching stores the intermediate attention states (the Key and Value vectors) from previous steps
- This turns quadratic recomputation into linear per-step work, the single biggest decode speed-up
- In Hugging Face XXXForCausalLM.generate(), this is the use_cache flag, True by default

The KV cache is linear in sequence length



Per-token KV memory is fixed, so total cache scales linearly with context length, and with batch size.

03

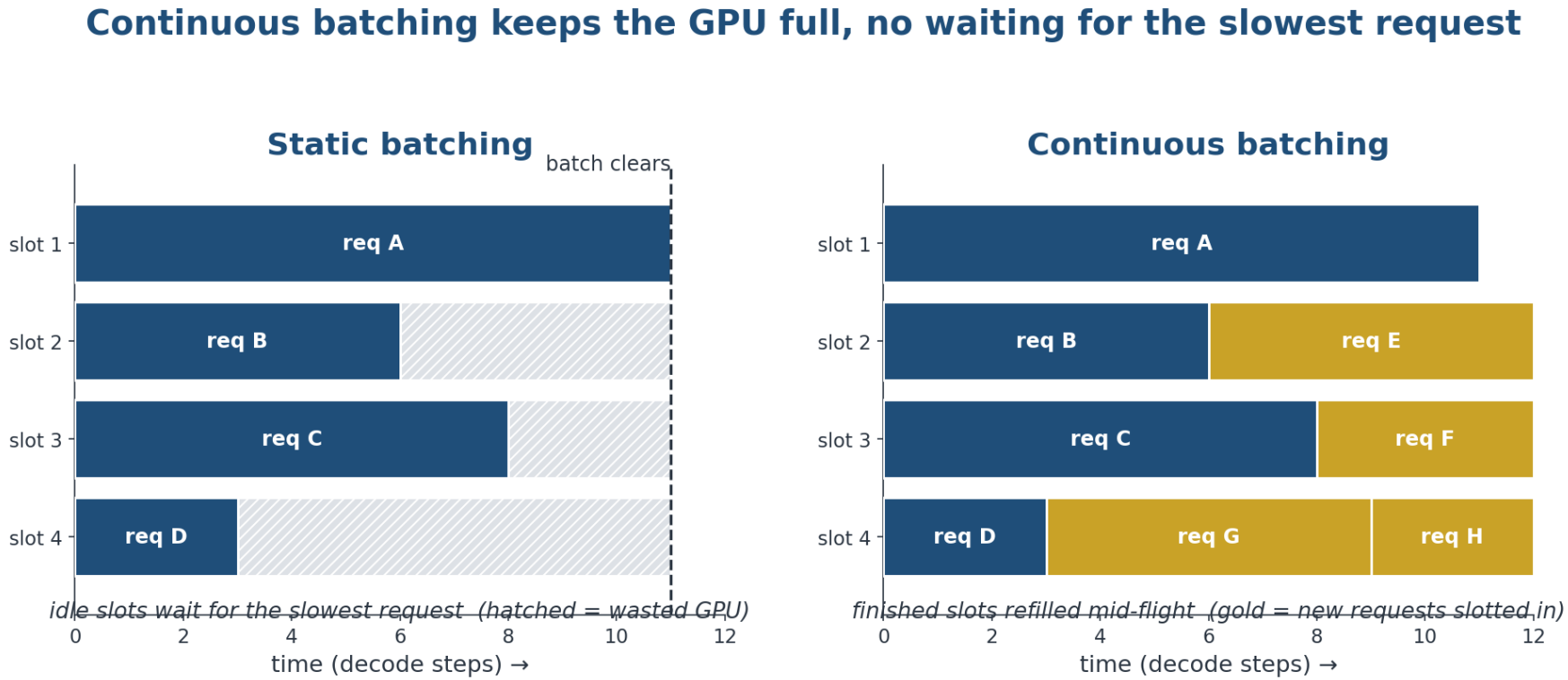
Serving Optimisations

The engineering that turns a slow, memory-bound decode into a high-throughput serving system.

Batching & continuous batching

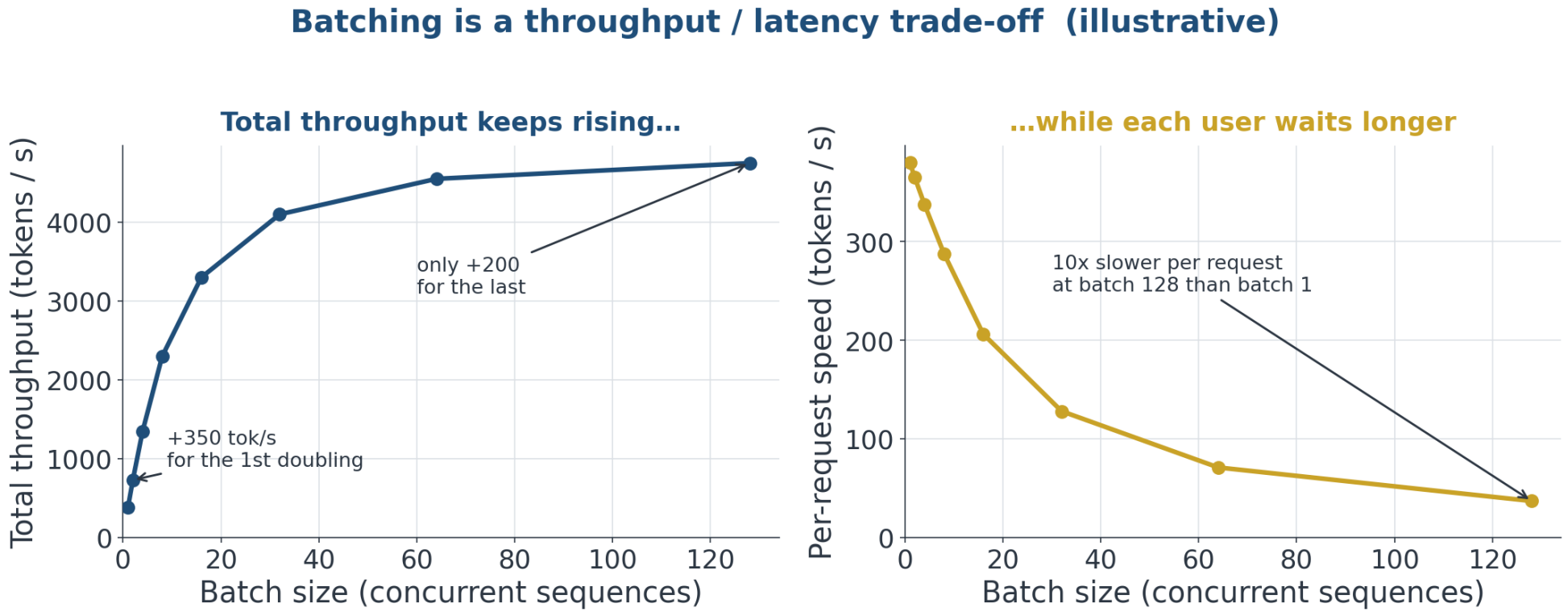
- Decode is memory-bound: each step reads all the weights (and the KV cache) to produce a single token, so the GPU's compute sits idle
- Batching fixes this: one read of the weights serves many requests at once, so the same memory traffic yields many tokens
- Larger batches use more of the GPU, but too large causes out-of-memory (OOM), the KV caches of all requests must fit
- Batching is hard because outputs vary wildly in length; some requests finish long before others
- Continuous batching (vLLM): the batch grows and shrinks dynamically during generation
- Finished sequences are evicted mid-flight and new requests slotted in, no waiting for the slowest

Static vs. continuous batching



Static batching leaves slots idle until the slowest request finishes; continuous batching evicts finished sequences and slots new ones in mid-flight, keeping the GPU full. Illustrative.

Why we batch, throughput vs batch size

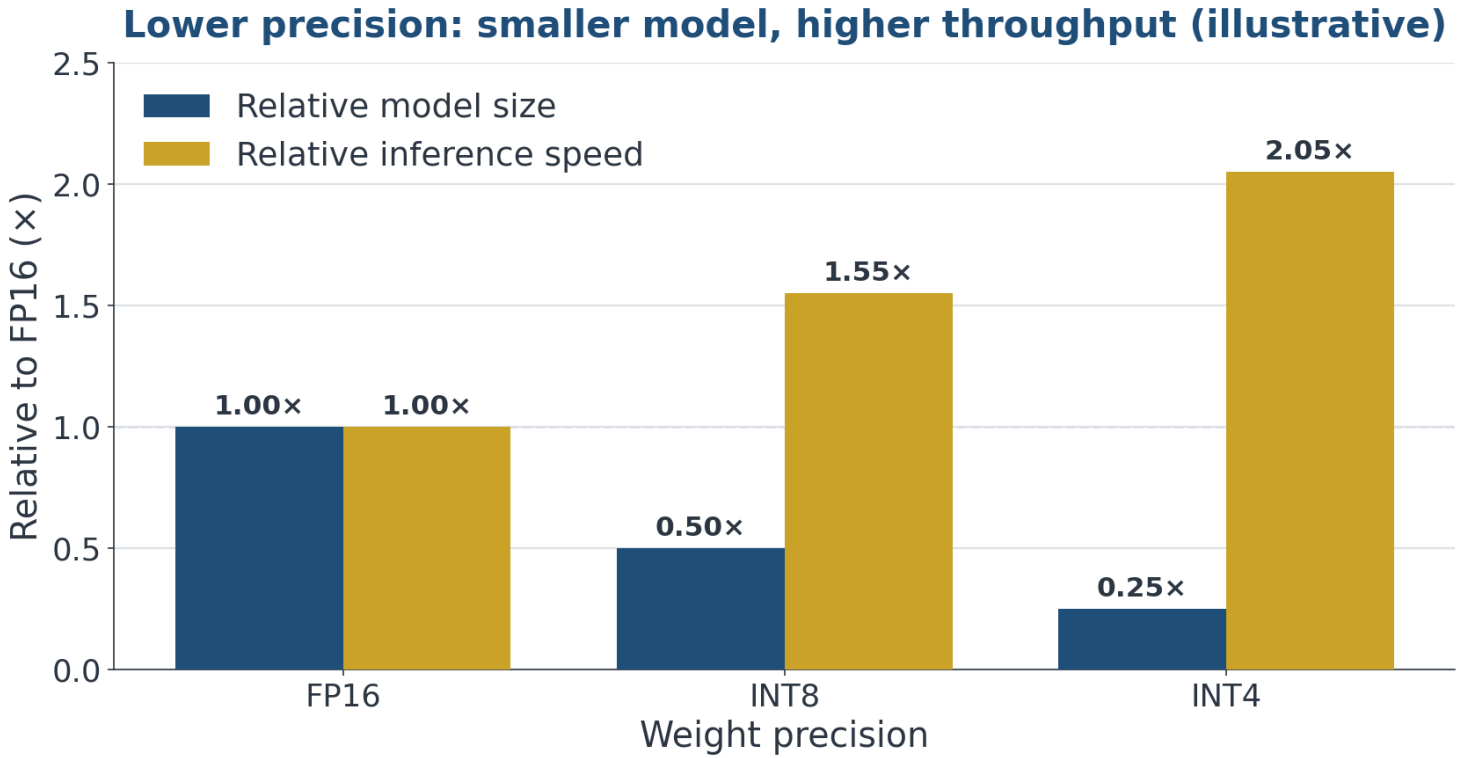


Total tokens/s keeps climbing, but each doubling buys less than the last, and per-request speed falls the whole way. Batch size is chosen against a latency target, not maximised. Illustrative.

Quantisation for inference

- Lowering weight/activation precision speeds inference and shrinks memory, the inference side of quantisation
- Post-training quantisation (PTQ): quantise after training/fine-tuning, purely to serve faster
- GPTQ is a common PTQ method, it uses a small calibration dataset to fit the quantisation parameters
- FP8 inference: 8-bit floating point on modern GPUs (Hopper/Blackwell), near-FP16 quality at higher throughput
- Frontier: binary (0,1) and ternary (−1,0,1) weight models push precision to the extreme

The quantisation trade-off



Dropping FP16 → INT8 → INT4 shrinks the model and lifts throughput, traded against some quality loss. Illustrative.

04

Serving Tools & Running Locally

The engines that package all of the above, and how to run a model on your own machine.

Model-serving engines

- vLLM: the de-facto open serving engine, continuous batching and efficient KV-cache memory management
- TGI (Hugging Face): easy no-code loading of Hub models, but now largely superseded by vLLM
- SGLang: high-throughput serving with aggressive prefix caching and structured output
- TensorRT-LLM (NVIDIA) and LMDeploy: compiler-optimised engines squeezing peak GPU performance
- Local devices: ollama and llama.cpp; compact model storage via GGUF

Serving engines, pick by workload

Engine	Best for	Throughput
vLLM	General GPU serving, continuous batching	Very high
SGLang	Heavy prefix reuse, structured / JSON output	Very high
NVIDIA Dynamo	Datacenter-scale, KV-aware routing, multi-node	Very high
TGI	Easy no-code loading of Hub models	High
Ollama / llama.cpp	Local & edge, CPU-friendly via GGUF	Modest

vLLM and SGLang lead server-side throughput; ollama / llama.cpp trade peak speed for local convenience.

Why run an LLM locally

- Privacy & security: no data leaves your machine
- Uninterrupted access: no rate limits or service outages
- Performance: possibly faster response, no network round-trip
- Customisation & cost: fine-tune freely; cheap if manpower isn't the bottleneck, plus offline inference
- ollama and llama.cpp are the common tools, ollama is a friendly wrapper over llama.cpp

GGUF, a portable model format

- A unified format that packs model parameters, architecture info and metadata compactly
- Designed to work across platforms; ollama uses GGUF for model compression
- Relies on quantisation: 1-, 2-, 3-, 4-, 5-, 6-, 8-bit variants, trading size for quality
- safetensors is another format, safe and fast, but stores weights without this packaging metadata

HANDS-ON

Walkthrough, run a model with ollama

- Download and install ollama on your computer, then open a terminal
- Browse available models on the ollama website / GitHub, small models come in 1B–4B sizes
  - ollama run qwen3:1.7b # pulls then runs a small current model
  - ollama pull qwen3:1.7b # pull only; run pulls automatically too
- Chat with no network latency, the response comes straight from your machine
  - ollama list # see local models · /bye to exit
- Needs enough RAM to download and run the model locally

HANDS-ON

Walkthrough, build a custom model (Modelfile)

- Create a Modelfile: a text file, stored where you'll call ollama, that customises a base model
- Give it a persona via a SYSTEM prompt and set decoding parameters:
  - FROM qwen3:1.7b
  - PARAMETER temperature 0.7 # sane sampling, NOT 5
  - SYSTEM """You are Mickey Mouse. Answer only as Mickey Mouse, the assistant."""
- Build and run it:
  - ollama create MickeyMouse -f ./Modelfile then ollama run MickeyMouse
- temperature 0.7 keeps replies coherent, a value like 5 would flatten the softmax into near-random text

05

Where Models Run

The same model can be served many ways, each a different cost, latency and privacy trade-off.

Types of LLM deployment

- API call to a provider, serverless: no infrastructure, pay per token
- Cloud-hosted: you host the model on a cloud provider's GPUs
- On-premises: your own local servers, inside your walls
- Edge: laptops, phones, IoT devices, the model runs on-device
- Hybrid: cloud + on-prem, or cloud + edge, split by workload or sensitivity
- Containerised: Docker / Kubernetes, portable packaging over any of the above

TRADE-OFFS

Deployment options at a glance

Option	Control	Cost model	Scalability	Privacy
Serverless API	Low	Per token	Instant / to zero	Data leaves
Managed cloud	Medium	Per GPU-hour	Provider-managed	Cloud-resident
On-prem	High	Capex + ops	You provision	Stays in-house
Local / edge	Full	Device hardware	Per device	On-device

Control and privacy rise as you move in-house; instant elastic scale is easiest to rent.

COST

The cost of serving LLMs

- API to a provider: token-based pricing, input and output tokens priced differently (output usually costs more)
- Cloud-hosted model: pay for compute time and GPU memory you reserve
- On-prem & edge: hardware, energy and manpower, capex plus ops
- Containerised: inherits the local or cloud cost model underneath it
- API-like cloud deploy: run an open or custom model behind an OpenAI-compatible API (e.g. Hugging Face Inference Endpoints, Modal, Replicate, Together AI)

EXAMPLE PRICING

Managed API vs. self-hosted open model

Provider API (per-token)

- Priced per 1M input / output tokens
- Zero infra; scales instantly to zero
- Output tokens cost more than input
- Great for spiky or low-volume workloads
- Data leaves your environment

Self-hosted open model (per-GPU-hour)

- Priced by GPU time + memory reserved
- You own scaling, batching & uptime
- Cheaper at high, steady throughput
- Full control of model & data residency
- Serve behind an OpenAI-compatible API

06

Evaluating LLMs

From serving a model to measuring it. No single number captures a model, you assemble a portfolio of evidence.

FRAMING

What we measure, two families

Intrinsic / automatic

- Perplexity, cross-entropy loss
- Reference-overlap (BLEU, ROUGE)
- Embedding similarity (BERTScore)
- Cheap, reproducible, fully offline
- But weakly correlated with real quality

Extrinsic / task & preference

- Standardized benchmarks (accuracy)
- Human ratings and LLM-as-judge
- Head-to-head arenas (Elo)
- Agentic task-completion suites
- Closer to what users care about

07

Reference-Based Metrics

BLEU, ROUGE and BERTScore: the classical toolkit, where it came from, and why it no longer fits.

BLEU, Bilingual Evaluation Understudy

- Invented at IBM Research (2001), originally for machine translation
- Computes n-gram precision of the model output against a reference text
- Counts n-grams in the generated text that also appear in the reference
- Adds a brevity penalty so short outputs can't game precision
- Score is precision-oriented: did the output stay close to the reference wording?

ROUGE, and its variants

Variant	What it overlaps
ROUGE-N	n-gram overlap
ROUGE-L	longest common subsequence (LCS)
ROUGE-W	weights contiguous matches more heavily
ROUGE-S	skip-bigram overlap (two words, not necessarily adjacent)

Compares overlapping units vs a reference, designed for summarization; recall-oriented: did the summary cover what the reference covered?

WORKED EXAMPLE

ROUGE by hand, Golden Gate Bridge

ROUGE Score
calculated by ngrams

well adapted in n grams era but
not GenAI

Reference describing the bridge

- Reference: 54 words describing the bridge; LLM summary: 31 words
- ROUGE-1 (unigrams): 22 matches → Precision 22/31 = 0.71, Recall 22/54 = 0.41, F1 = 0.52
- ROUGE-2 (bigrams): 15 matches → Precision 0.50, Recall 0.28, F1 = 0.36
- ROUGE-L (LCS): e.g. ABCDEFG vs BACDYETTT → LCS = CDE; here LCS = 21 words, Precision 0.68
- Notice: a correct paraphrase with different words is punished, the metric only sees overlap

BERT model gives the embedding

more adapted to GEN AI, using BERT model to combine embedding cosine similarity (b/n reference embedding & summary embedding)

BERTScore, similarity in embedding space

- BERTScore (Zhang et al., 2020) replaces string overlap with similarity of meaning
- Embed every token of the candidate and of the reference with a pre-trained encoder (BERT, Topic 2)
- Match each candidate token to its most similar reference token by cosine similarity
- Average the matches: candidate → reference is precision, reference → candidate is recall, combine to F1
- "The bridge is red" vs "The bridge is crimson" scores high: red and crimson have close embeddings

lives close in the embedding space

- Limits: still needs a reference, depends on the encoder, and a fluent wrong answer can still score well

summarisation text —> do we have a reference text? —> the article uw to be summarised

THE CATCH

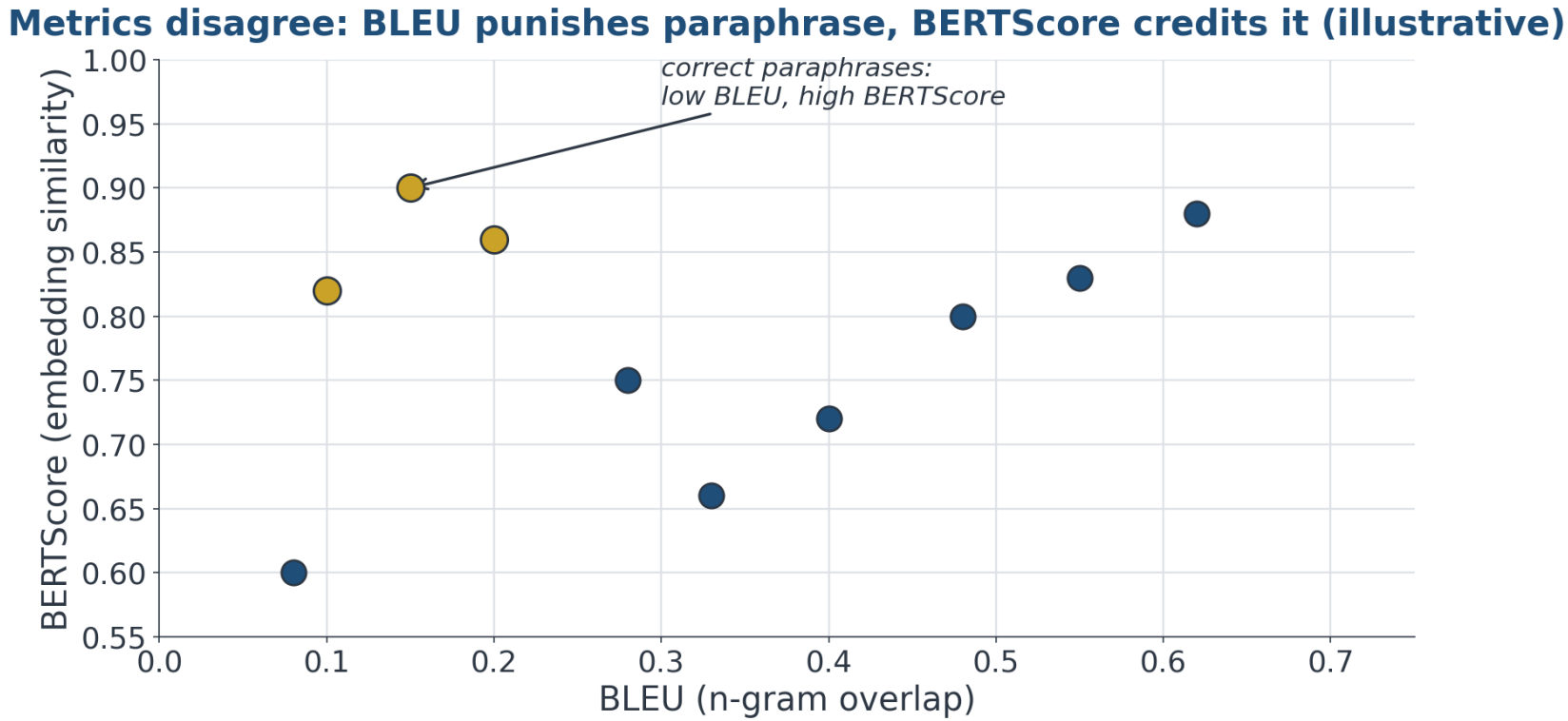
looks for word overlap, not good for LLM as it does not keep using the same words

Do NOT score LLM generation with BLEU/ROUGE

- They only count exact string / n-gram overlap with a reference text
- A correct paraphrase in different words gets no credit, the metric cannot see meaning
- Open-ended generation has no single correct reference, so overlap with one is arbitrary
- Their scores correlate weakly with human judgment on LLM output
- Common project mistake: reporting BLEU / ROUGE as the quality metric for a chatbot loses marks

METRIC DISAGREEMENT

Why overlap metrics mislead on paraphrase



Correct paraphrases score low on BLEU (n-gram overlap) yet high on BERTScore (embedding similarity), the metrics rank the same outputs differently. Illustrative.

METRIC FAMILIES

Text-quality metrics at a glance

Metric	What it compares	Good for / limitation
BLEU	n-gram precision vs reference	MT; blind to paraphrase
ROUGE	n-gram / LCS overlap	summarization; overlap-only
BERTScore	embedding similarity	credits correct paraphrase
Task metrics	exact match / unit tests	verifiable tasks; needs a checker

Overlap metrics (BLEU/ROUGE) are inadequate for open-ended generation; prefer semantic or task metrics, and the judges we meet later.

08

Perplexity, Entropy & Cross-Entropy

The intrinsic metric that falls straight out of the training loss.

INTUITION

Perplexity, the model's own uncertainty

measures avg uncertainty of gen model predicting the next word.
it is a measure of entropy (e.g how predictable next word is given the context)

difference between cross entropy and entropy?
-cross entropy is comparing 2 probability distribution (ground truth & the model output)
-entropy is looking at one distribution on its own (output distribution sharp —> entropy is low) ; (Output distribution flat —> entropy is high)

- PPL, first introduced 1977 by IBM researchers on speech recognition
- Quantifies the average uncertainty of the model's next-token predictions
- Entropy (Shannon, 1950): how unpredictable the next word is given context
- Lower entropy = more predictable language; LLM output is a distribution, so entropy is easy to compute
- PPL ≈ the average number of options the model finds plausible per token, lower is more confident

WORKED EXAMPLE

Perplexity, a worked example

- Take a 4-token sentence and look only at the probability each model gave the *true* next token

Model A sharper distribution than B. but it does not tell u which model is the correct one as we dont have the ground truth.

- Model A: 0.5 at every step. Mean cross-entropy = −ln 0.5 = ln 2, so PPL = exp(ln 2) = **2**

model A is more certain than model B.

- Model B: 0.25 at every step. Mean cross-entropy = ln 4, so PPL = exp(ln 4) = **4**
- Read it as "plausible options per token": A is choosing between 2, B between 4
- Equal probabilities make it exact; with mixed probabilities PPL is their geometric mean
- Lower is better, and PPL = exp(cross-entropy loss), so it falls out of training for free

IN PRACTICE

Computing PPL, and its traps

- In PyTorch it's one line: perplexity = torch.exp(loss) on the cross-entropy loss
- Caution: PPL is confidence, not correctness, a model can be confidently wrong or correctly unsure
- Caution: not comparable across models, tokenization, vocab size, dataset and context length all shift it
- Best use: track PPL of one model on one dataset (e.g. before vs after fine-tuning)

09

Knowledge & Reasoning Benchmarks

Standardized, comparable, and constantly being saturated.

LANDSCAPE

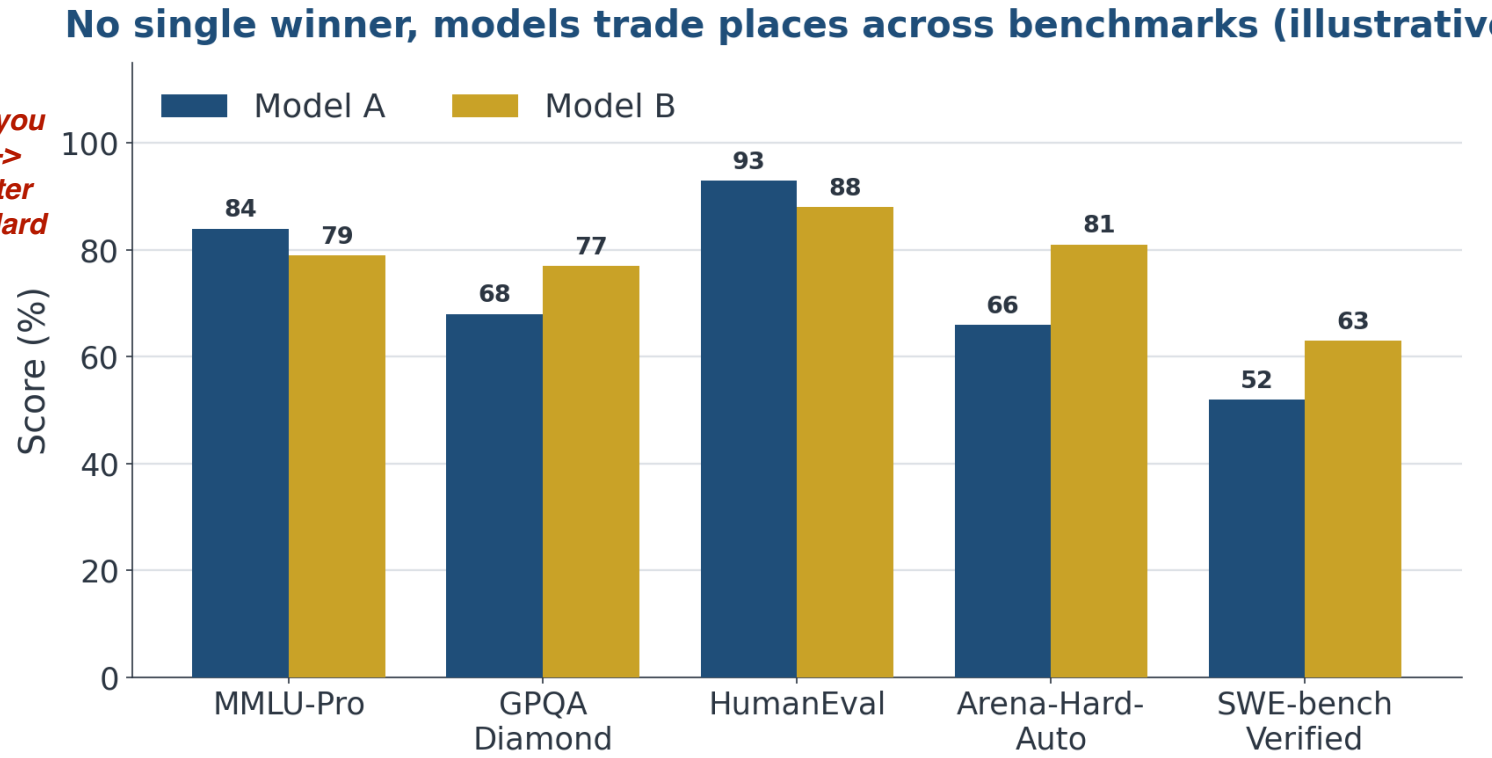
The benchmark landscape

Benchmark	What it measures	Format / grading
MMLU-Pro	broad knowledge & reasoning	10-option MCQ
GPQA Diamond	expert science reasoning	'Google-proof' MCQ
HumanEval	code generation	unit tests (pass@k)
Arena-Hard-Auto	open-ended chat quality	LLM-as-judge, auto
SWE-bench Verified	agentic bug-fixing	repo test suite

No single benchmark covers a model, a scorecard spans knowledge, reasoning, code, chat and agentic tasks.

One scorecard, many axes, no single winner

when you finetune model you are changing the weights -> must compare before & after fine tune by running standard benchmarks



The same two models trade places across benchmarks: A leads on broad knowledge and code, B on expert science, chat quality and agentic coding. Which is 'better' depends on what you weight. Illustrative.

SATURATION

Benchmarks saturate, then get replaced

older benchmark easier as model quality getting better

data leakage: benchmark themselves gets into the training data

- Benchmarks have a shelf life: MMLU (2021, 57 subjects, 4-option MCQ) was hard at ~60%
- Frontier models now score above 90% on it, so it no longer separates them, everyone sits at the ceiling
- Saturation forces a harder successor: MMLU → MMLU-Pro (10 options, harder items), GPQA Diamond → HLE
- The same fate awaits every public benchmark, the shelf life is a few years at most
- Rule: never evaluate a new model on an old benchmark; use the current suite and say which version you ran

CONTAMINATION

Benchmarks also leak, data contamination

- Old public benchmarks end up in pre-training data, then the score measures memorisation, not ability
- Symptom: suspiciously high scores that collapse on paraphrased or freshly-written variants of the items
- Defences: rolling or held-out sets, canary strings in the data, contamination audits before publishing a score
- Prefer newer or private eval sets for headline claims; treat old public benchmarks with suspicion
- This is why leaderboards keep moving, and why you should build a small private eval for your own task

Math, reasoning you can grade automatically

- Math questions are easy to source and grade, the answer is right or it isn't
- Tests reasoning / chain-of-thought: the method must be right, not just the final number
- Hendrycks MATH: 12,500 competition problems, the de-facto standard; MetaMath is another good set
- AIME (2025/2026): the headline competition-math benchmark every frontier reasoning-model scorecard now quotes
- Caveat: many problems leaked into pre-training data, a real contamination risk (previous slides)

SCIENCE

Science, GPQA Diamond

- Older sets (e.g. ScienceQA) are now too easy for frontier models
- GPQA Diamond: 'Google-proof' Q&A, 448 expert MCQs in biology, physics, chemistry
- PhD-level experts reach ~65–75%; non-experts with open web access reach only ~34%
- Designed so the answer can't be found by a quick search, it probes genuine understanding
- Shelf-life: GPQA Diamond is saturating too, like MMLU before it
- Successor: Humanity's Last Exam (HLE), ~2,500 expert questions across domains

10

Coding & Long-Context Benchmarks

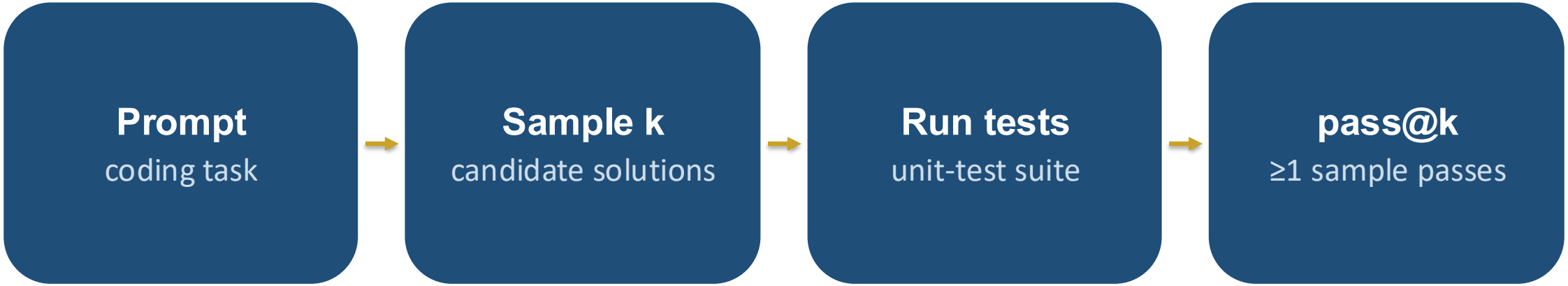
Two capabilities that are both high-value and hard to measure.

Coding benchmarks & pass@k

- Coding assistance is a flagship use case, driving many code-specialized models
- Tasks: completion, explanation, bug-fixing, translation between languages
- HumanEval and MBPP are the classic sets; graded by running unit tests, not string match
- pass@k: probability that at least one of k sampled solutions passes the tests
- LiveCodeBench is newer and contamination-resistant (rolling problems, an 'execute' task)

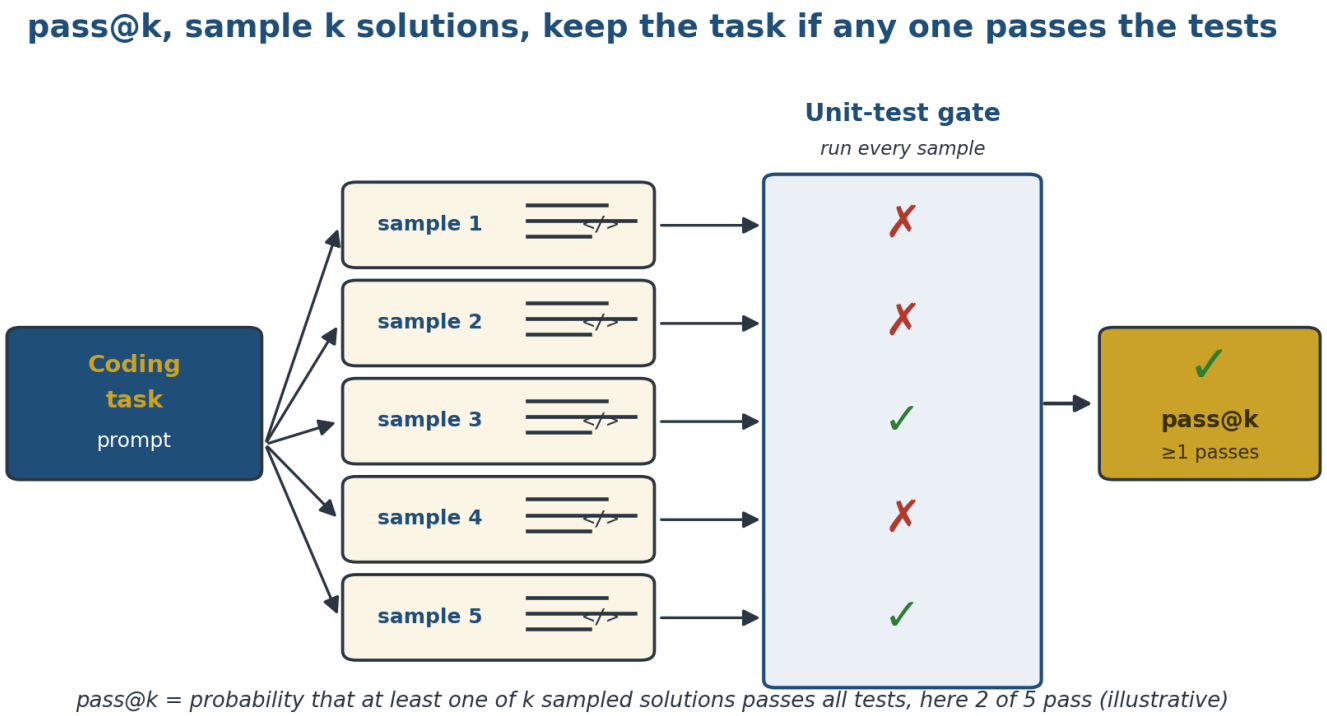
GRADING

How pass@k grading works



Graded by execution, not string match, pass@k is the chance at least one of k sampled solutions passes all tests.

pass@k, illustrated, the test gate decides



k independent samples are each executed against the unit-test suite; the task counts as solved if at least one passes, here 2 of 5. Illustrative.

Long-context evaluation

token context: size of input, 1M token means put entire 1M in my prompt and ask questions about it

is model correctly using all the 1M context it is suppose to use ?

- Models now accept 128K–1M+ token contexts, but do they actually use all of it?
- Needle-in-a-haystack / passcode retrieval: hide one fact in a huge context and ask for it
- Frontier models largely pass simple needle tests, retrieval is nearly solved at that level
- Harder multi-fact / reasoning-over-context tasks exist but are compute-expensive to run

11

Human & LLM-as-Judge

For open-ended quality, you need a judge, human, model, or the crowd.

Programmatic checks, verify the output in code

Check	How it works	Example
Exact match	compare the output string to the expected answer	"Capital of France?" → "Paris"
Rule / regex	test the output against a pattern or rule	reply contains a valid email address
Schema adherence	parse the output, validate required fields & types	extraction returns well-formed JSON
Numeric field	extract a number, compare within a tolerance	invoice total = 42.00 ± 0.01
Unit tests	execute the output, run a test suite (pass@k)	generated function passes all tests

A programmatic check is code, not a model: deterministic, cheap, unbiased and reproducible. It is the first thing to try.

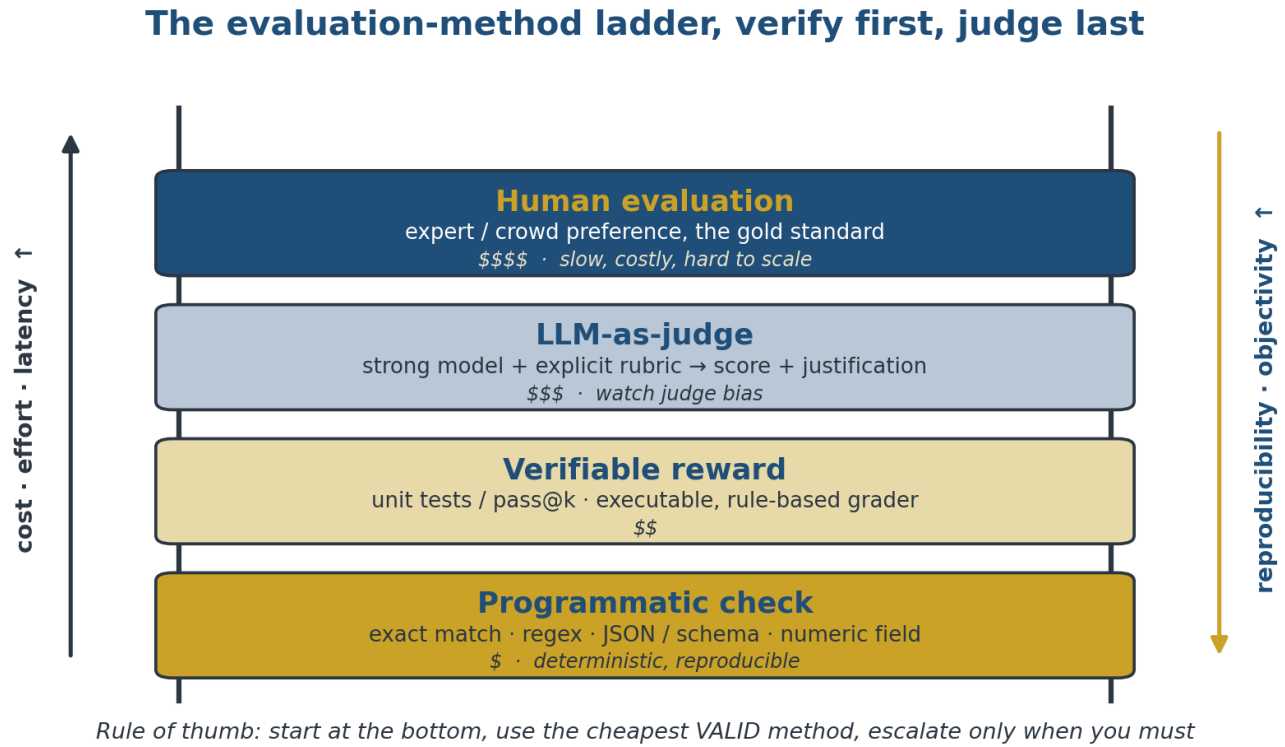
FIRST PRINCIPLE

Verify first, judge only what you can't check

- Before reaching for a judge, ask: can I check this output in code? If yes, use a programmatic check
- Verifiable reward: the same idea packaged as an executable grader that returns a score (unit tests, a rule-based checker)
- This is why coding benchmarks are execution-graded (HumanEval), not string-matched
- Reserve LLM-as-judge for the open-ended part: helpfulness, tone, reasoning quality
- Human evaluation is the gold standard, and the most expensive, so it comes last
- Cheapest valid method wins: programmatic check → verifiable reward → LLM-judge → human

THE LADDER

Verify first, judge last, the method ladder



Climb only as high as you must: the bottom rungs are cheap, deterministic and reproducible; each step up adds cost and subjectivity. Illustrative.

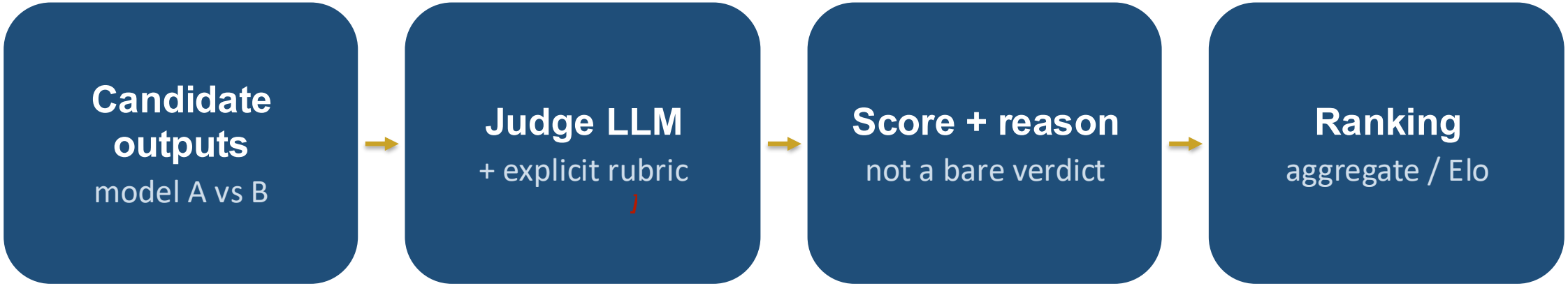
JUDGES

Judging open-ended quality

- Response quality matters most but is exactly what accuracy benchmarks miss
- Human judges are the gold standard, but slow and expensive at scale
- LLM-as-judge: a strong model scores or compares outputs against a rubric, effective and cheap
- MT-Bench (2023) was the origin of automatic LLM-as-judge; Arena-Hard-Auto (v2) is the current auto-benchmark
- Practical note: judging with closed models needs an API key to the judge

JUDGE

The LLM-as-judge pipeline

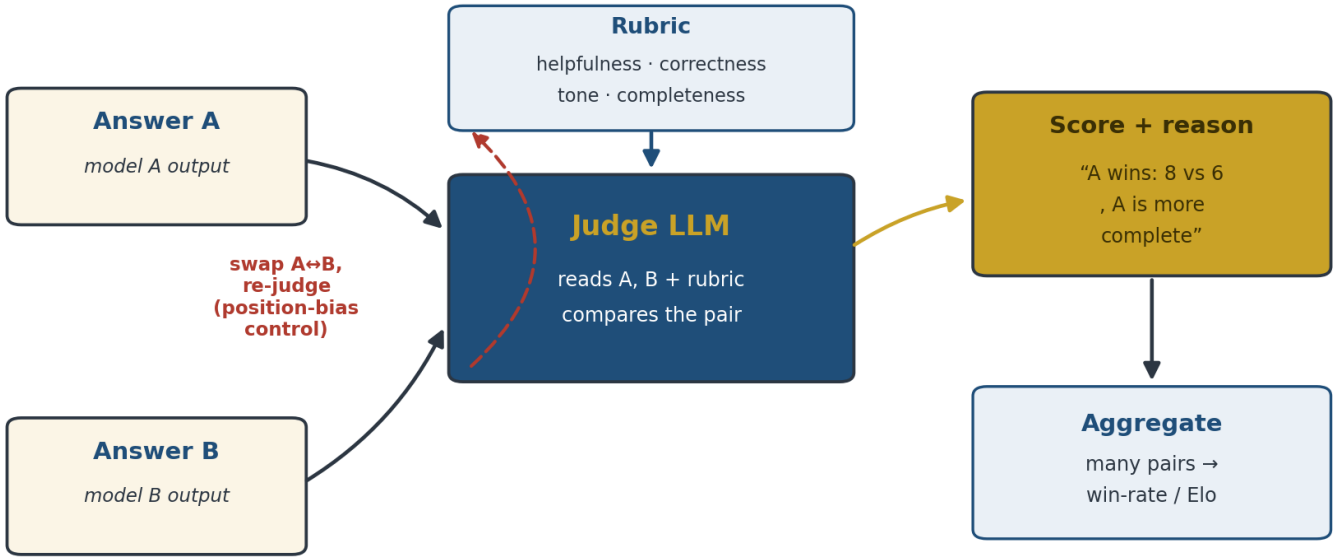


Validate the judge against human labels and swap answer order to control position bias (see next slide).

LLM-as-judge: rubric, scores, swap check

hallucinate, not reproducible

LLM-as-judge pipeline, rubric-scored pairs, with a position-swap check



Two candidate answers are scored against an explicit rubric; the judge returns a score plus justification, aggregated to a win-rate/Elo. Swapping A↔B and re-judging controls position bias. Illustrative.

JUDGE DESIGN

LLM-as-judge, implementation & bias

- Give the judge an explicit rubric and ask for a score + justification, not a bare verdict
- Known biases: position bias (favours first answer), verbosity bias, self-preference (rates its own family higher)
- Mitigations: randomize / swap answer order, control for length, use a different-family judge, ensemble judges
- Always validate the judge against a human-labeled sample before trusting it at scale
- A biased judge produces confident, reproducible, wrong numbers, the worst kind

12

A Frontier Case Study

How to read a current leaderboard, and why evaluation is now the bottleneck.

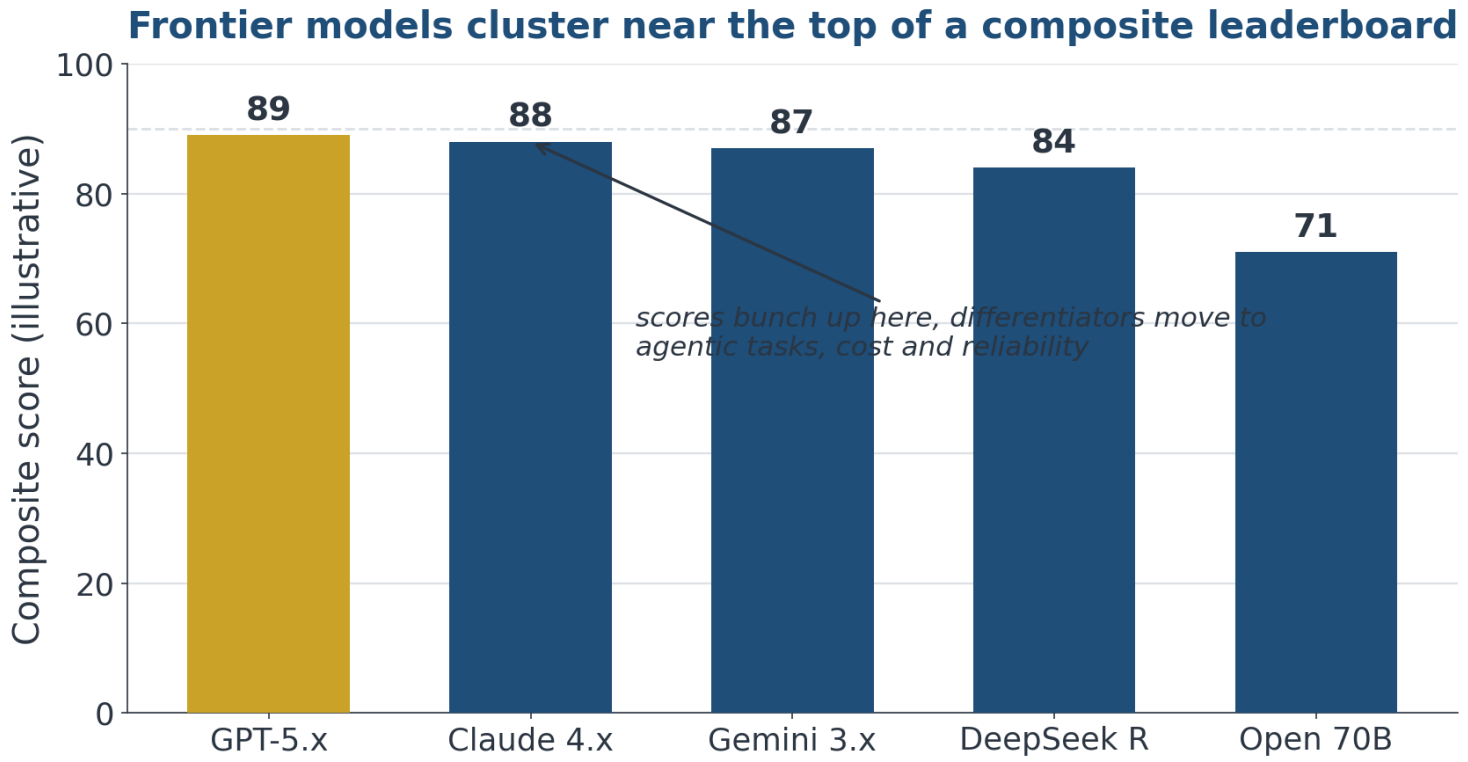
CASE STUDY

Reading a current frontier leaderboard

- Assess a model against many peers across a spread of eval sets, never a single number
- 2026 frontier class: GPT-5.x, Claude 4.x, Gemini 3.x, and DeepSeek R-series (reasoning models)
- Typical scorecard: MMLU-Pro + GPQA Diamond + HLE (knowledge), AIME + MATH (reasoning), SWE-bench Verified (agentic coding), Arena Elo (preference)
- Read critically: check few-shot vs zero-shot, whether reasoning/thinking was on, and contamination controls
- Scores cluster near the top, the differentiators are now agentic tasks, cost, and reliability

LEADERBOARD

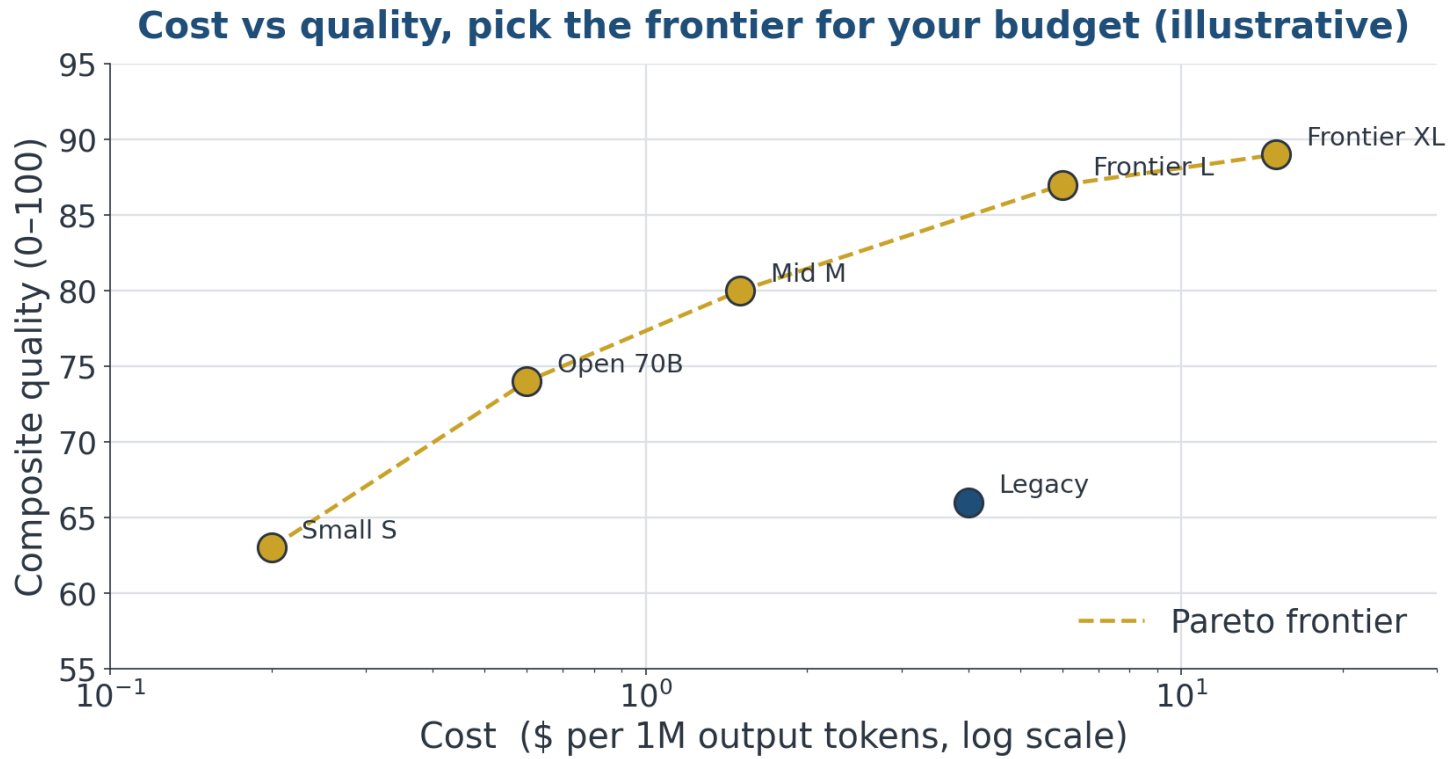
A frontier leaderboard, at a glance



Illustrative composite scores, frontier models bunch near the top, so agentic tasks, cost and reliability differentiate. Illustrative, not official.

COST VS QUALITY

Quality isn't the only axis, cost matters



Plotting cost against quality exposes a Pareto frontier, the best model for a budget, not the single 'best' model. Illustrative, not official.

Evals are the bottleneck

- Frontier progress is increasingly gated by measurement, not compute, you can't improve what you can't measure
- Good benchmarks saturate fast and leak, so authoring hard, clean evals is itself hard research
- Agentic evals are expensive: multi-step rollouts with tool calls cost real money and time to run
- A private, task-specific eval harness is a genuine competitive advantage, it is your ground truth
- Takeaway: treat evaluation as a first-class artifact, not an afterthought

Key takeaways

01

Match deployment to the workload

Serverless vs managed cloud vs local/on-prem, trade cost, latency, control and privacy.

02

Inference efficiency is engineering

KV cache, continuous batching and quantisation drive cost & latency.

03

Measure the right thing

BLEU/ROUGE are inadequate for generation, prefer BERTScore, LLM-as-judge and task metrics.

04

Benchmarks age and leak

Use current suites (MMLU-Pro, agentic + safety evals), watch contamination, build a task-specific set.